Assignment 3: Stock Price, Return and Cumulative Return Analysis

Name: Danis Shrestha

USN: 25MBAR0482

Section: FBA-3

Sector: Footwear | **Stocks:** Bata India, Relaxo Footwears, Campus Activewear, Metro Brands, Liberty Shoes **Period:** 1 October 2025 to 30 September 2026 (one year)

1. Download the data and store it in a DataFrame

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import yfinance as yf

stocks = ["BATAINDIA.NS", "RELAXO.NS", "CAMPUS.NS", "METROBRAND.NS", "LIB
start_date = "2025-10-01"
end_date = "2026-10-01"      # yfinance excludes the end date, so the las

stock_data = {}
for symbol in stocks:
    stock_data[symbol] = yf.download(symbol, start=start_date, end=end_da
                                     multi_level_index=False, progress=Fa
    print(symbol, "rows:", len(stock_data[symbol]))
```

```
BATAINDIA.NS rows: 247
RELAXO.NS rows: 247
CAMPUS.NS rows: 247
METROBRAND.NS rows: 246
LIBERTSHOE.NS rows: 246
```

In [2]:
```
# Combine everything into one DataFrame (one block per stock)
dfs = []
for symbol, data in stock_data.items():
    data["Stock"] = symbol
    dfs.append(data)
stocks_df = pd.concat(dfs)
print(stocks_df.head())
print(stocks_df.tail())
```

```
                 Close         High          Low         Open  Volume  \
Date
2025-10-01  1133.892090  1139.332093  1111.368880  1122.344184  147367
2025-10-03  1156.129150  1178.843191  1134.083125  1134.083125  212147
2025-10-06  1164.623047  1173.689603  1146.203576  1146.203576  106582
2025-10-07  1162.523438  1181.515511  1151.929905  1166.245521   82257
2025-10-08  1146.203613  1166.245475  1140.668195  1162.905165   45927

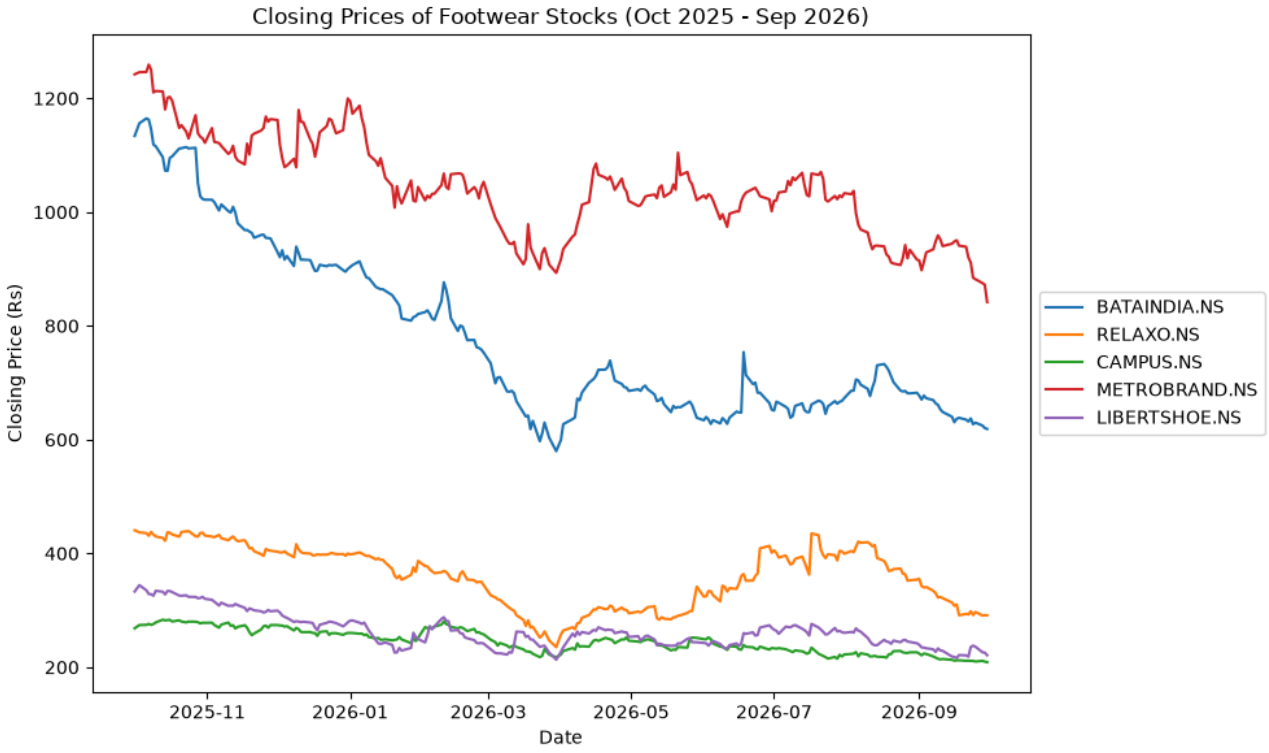
                  Stock
Date
2025-10-01  BATAINDIA.NS
2025-10-03  BATAINDIA.NS
2025-10-06  BATAINDIA.NS
2025-10-07  BATAINDIA.NS
2025-10-08  BATAINDIA.NS
                Close        High         Low        Open  Volume  \
Date
2026-09-24  237.839996  242.339996  235.000000  241.399994   45513
2026-09-25  235.919998  242.000000  234.500000  240.800003   21983
2026-09-28  226.199997  239.589996  225.520004  231.179993   19650
2026-09-29  225.539993  229.059998  221.050003  229.059998   15992
2026-09-30  221.009995  229.000000  220.460007  229.000000   30337

                   Stock
Date
2026-09-24  LIBERTSHOE.NS
2026-09-25  LIBERTSHOE.NS
2026-09-28  LIBERTSHOE.NS
2026-09-29  LIBERTSHOE.NS
2026-09-30  LIBERTSHOE.NS
```

2. Closing prices of all 5 stocks in one chart

In [3]:

```python
plt.figure(figsize=(10, 6))
for symbol, data in stock_data.items():
    plt.plot(data.index, data["Close"], label=symbol)
plt.title("Closing Prices of Footwear Stocks (Oct 2025 - Sep 2026)")
plt.xlabel("Date")
plt.ylabel("Closing Price (Rs)")
plt.legend(loc="center left", bbox_to_anchor=(1, 0.5))
plt.tight_layout()
plt.show()
```



3. Daily returns (stored in a DataFrame)

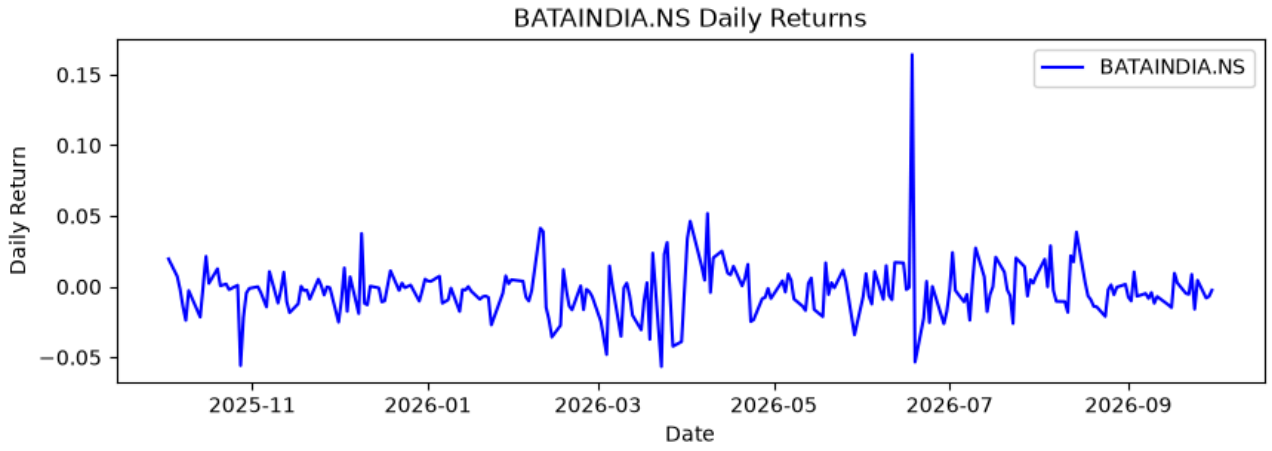
In [4]:
```
closing = pd.DataFrame({s: d["Close"] for s, d in stock_data.items()})
returns_df = closing.pct_change()          # (today / yesterday) - 1
print(returns_df.head())
```

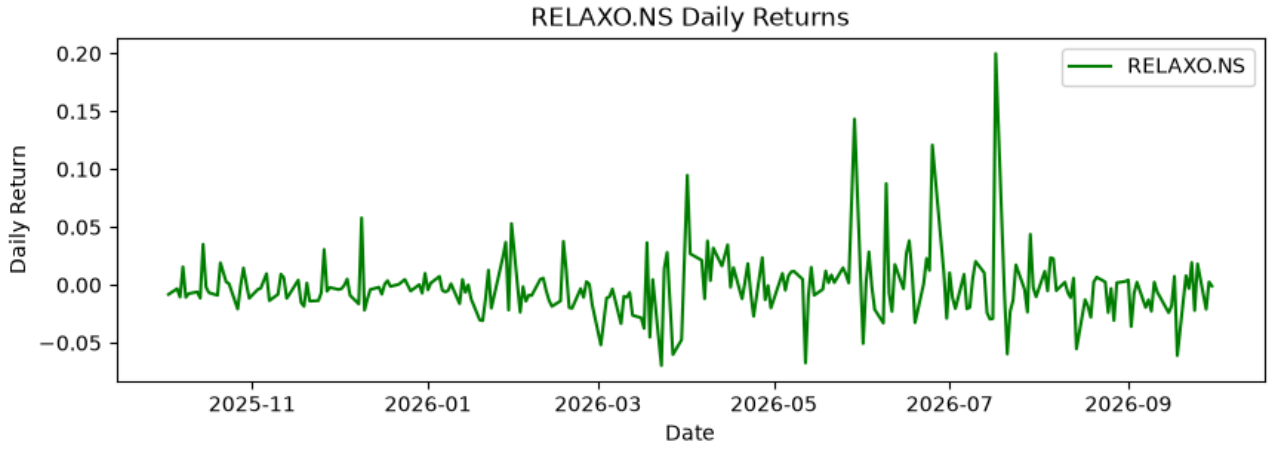
```
            BATAINDIA.NS  RELAXO.NS  CAMPUS.NS  METROBRAND.NS  LIBERTSHO
E.NS
Date
2025-10-01           NaN        NaN        NaN            NaN
NaN
2025-10-03      0.019611  -0.008191   0.021470       0.003040       0.033
483
2025-10-06      0.007347  -0.003281   0.000906       0.000319      -0.026
297
2025-10-07     -0.001803  -0.010556   0.006517       0.010367      -0.019
397
2025-10-08     -0.014038   0.015602  -0.006475      -0.007025      -0.000
913
```

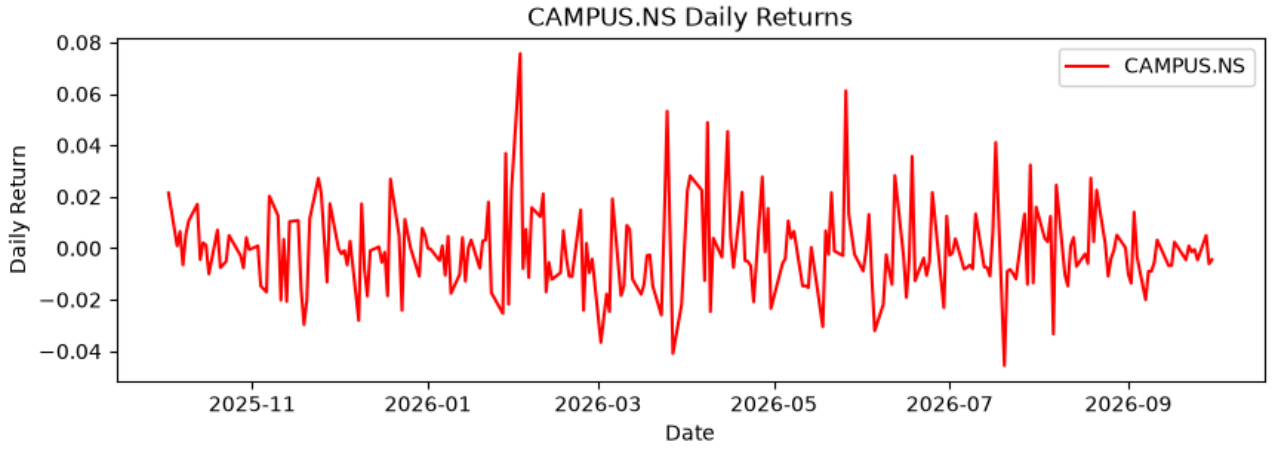
4. Daily returns, plotted separately

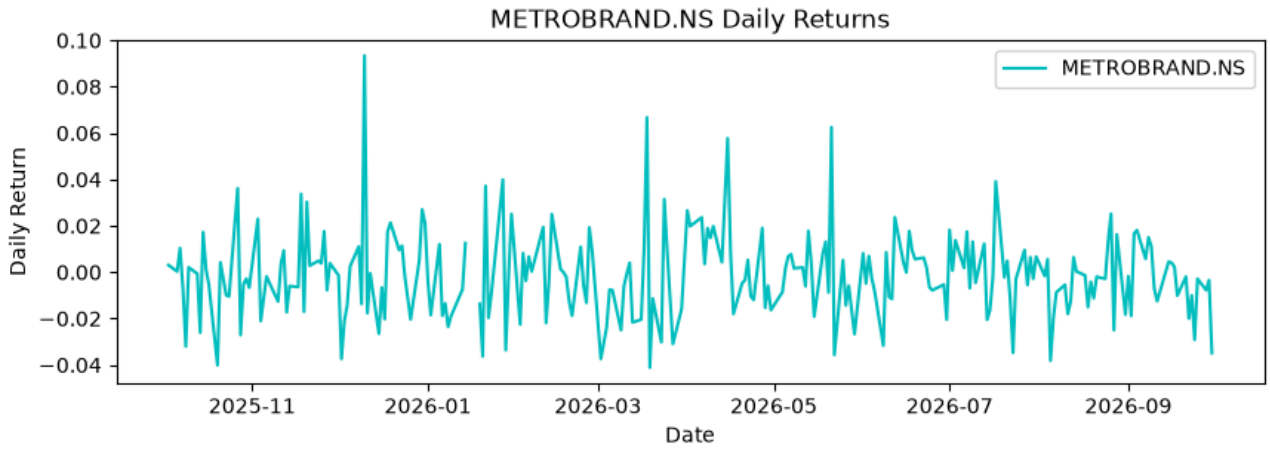
In [5]:
```
#  a different colour for each stock
colors = ["b", "g", "r", "c", "m"]
for i, symbol in enumerate(returns_df.columns):
    plt.figure(figsize=(10, 3))
    plt.plot(returns_df.index, returns_df[symbol], color=colors[i], label
```

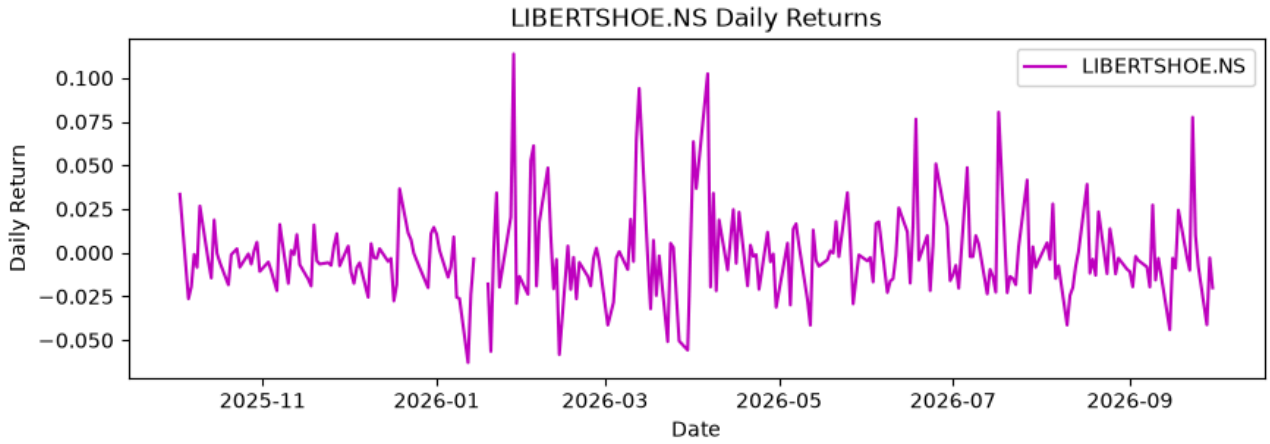
```
    plt.title(f"{symbol} Daily Returns")
    plt.xlabel("Date")
    plt.ylabel("Daily Return")
    plt.legend()
    plt.show()
```











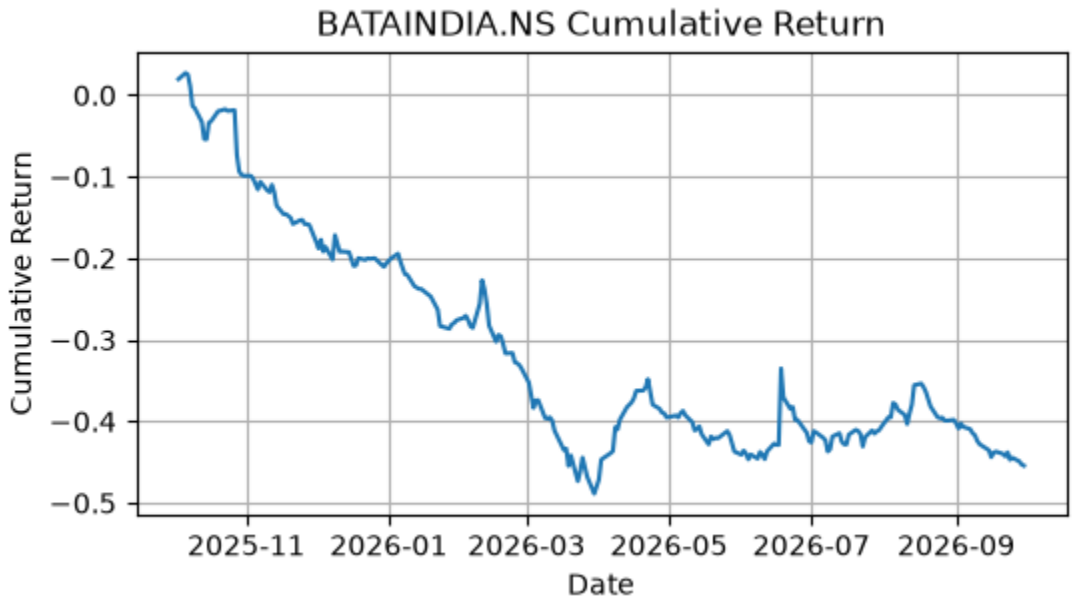
5. Cumulative returns (stored in a DataFrame)

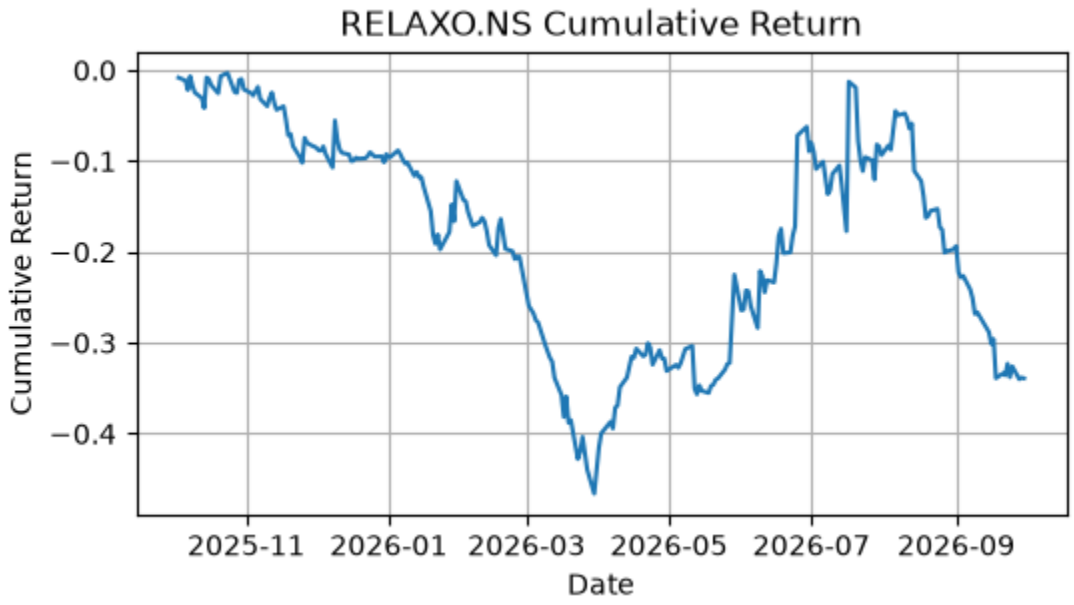
In [6]:
```
cumulative_returns_df = (1 + returns_df).cumprod() - 1
print(cumulative_returns_df.tail())
```

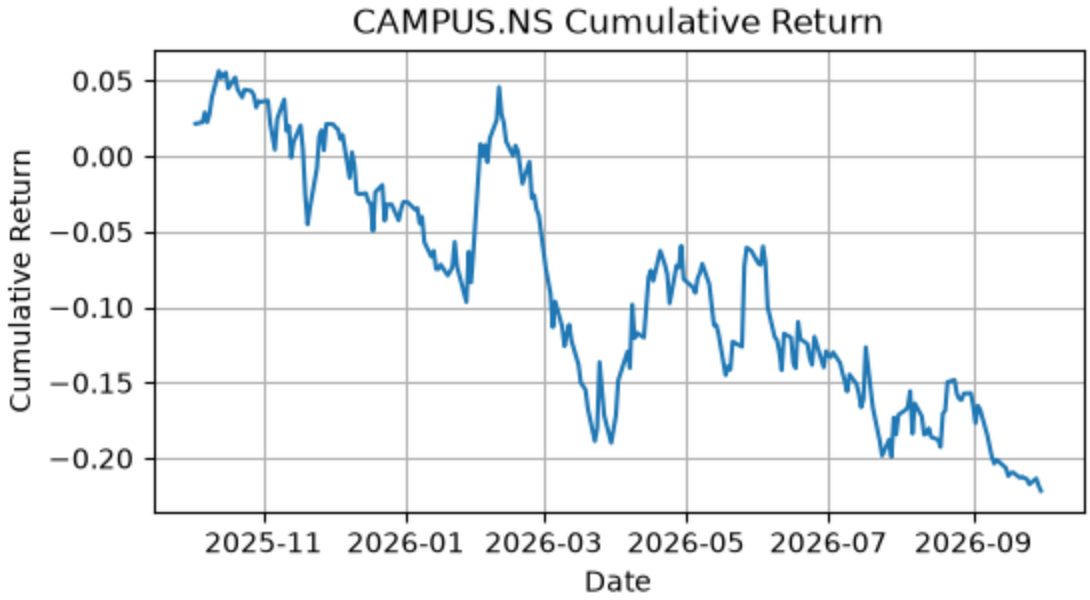
```
            BATAINDIA.NS  RELAXO.NS  CAMPUS.NS  METROBRAND.NS  LIBERTSHO
E.NS
Date
2026-09-24     -0.447037  -0.337991  -0.213786      -0.264726      -0.293
100
2026-09-25     -0.444568  -0.325963  -0.217363      -0.266845      -0.298
807
2026-09-28     -0.449066  -0.340147  -0.213450      -0.272539      -0.327
696
2026-09-29     -0.452902  -0.338445  -0.218221      -0.275033      -0.329
658
2026-09-30     -0.454313  -0.339126  -0.221687      -0.300303      -0.343
122
```

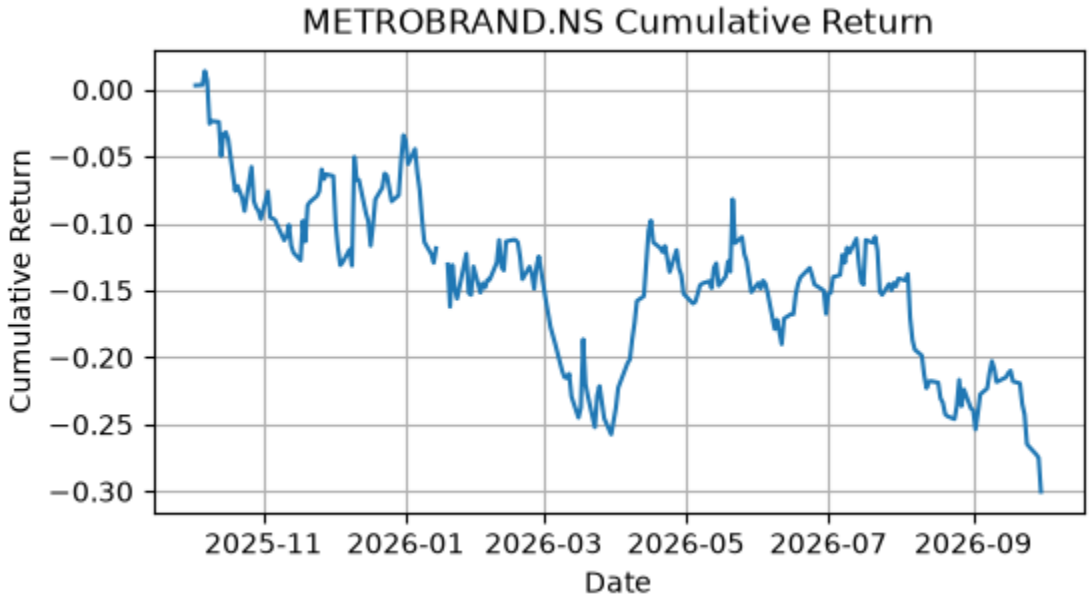
6. Cumulative returns: individual, combined and grid of subplots

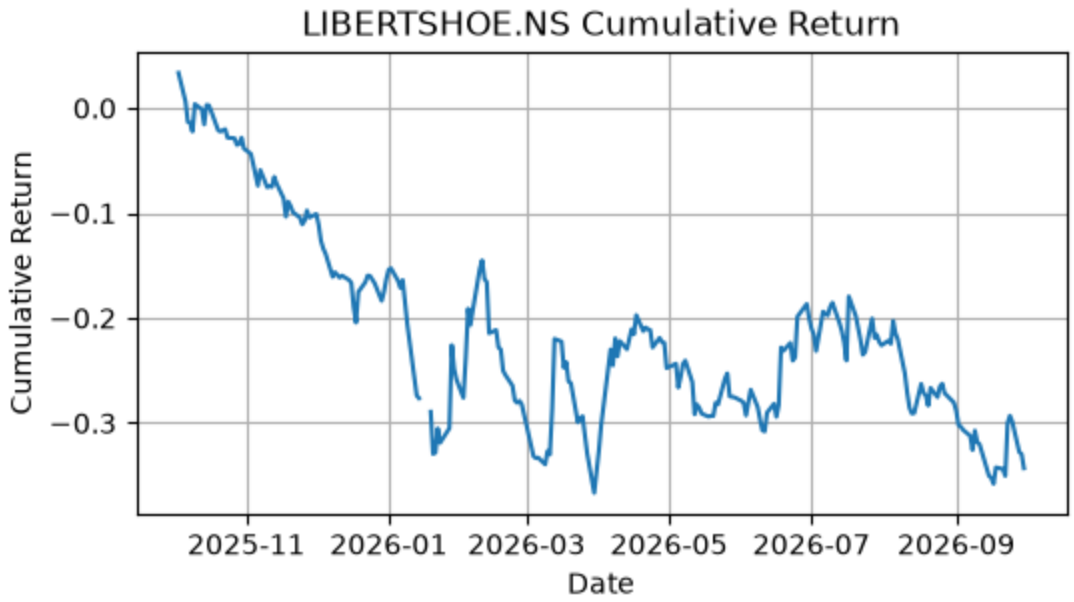
In [7]:
```
# Individual charts
for symbol in cumulative_returns_df.columns:
    plt.figure(figsize=(6, 3))
    plt.plot(cumulative_returns_df.index, cumulative_returns_df[symbol])
    plt.title(f"{symbol} Cumulative Return")
    plt.xlabel("Date")
    plt.ylabel("Cumulative Return")
    plt.grid(True)
    plt.show()
```



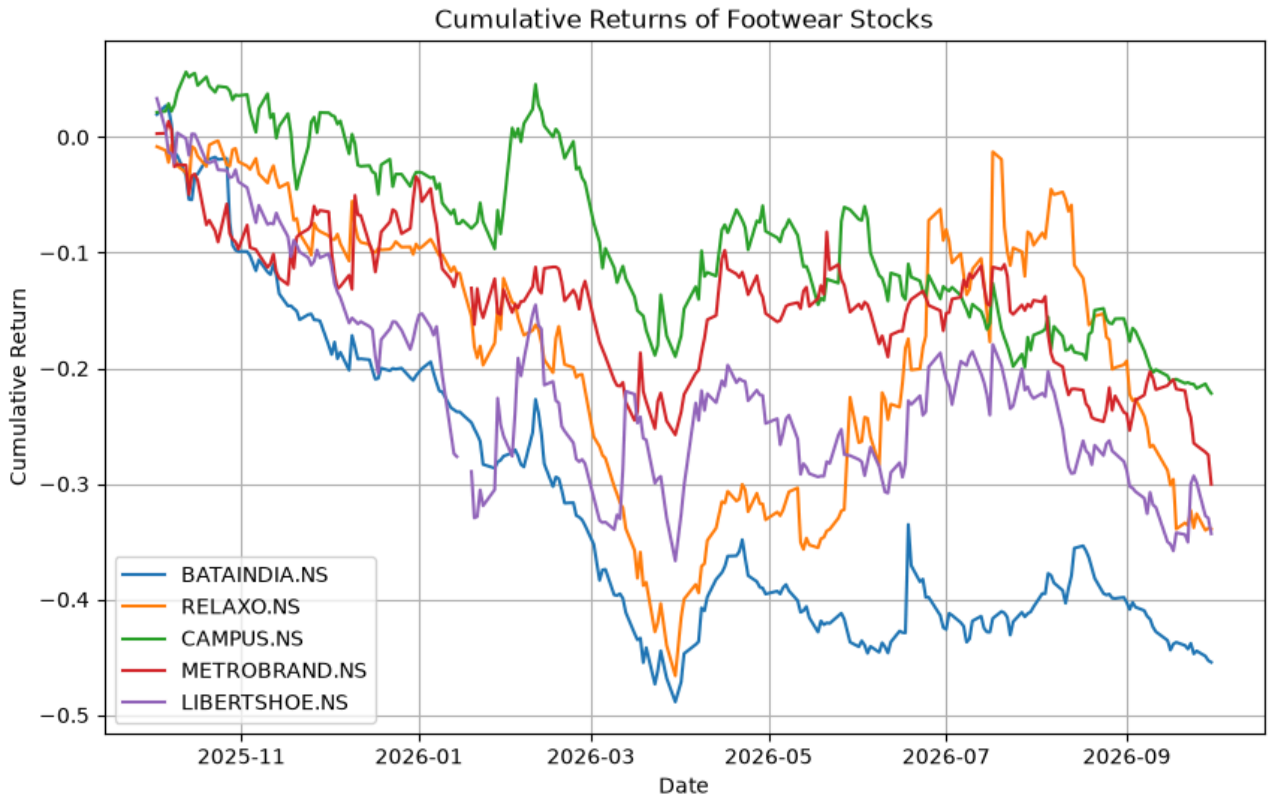



CAMPUS.NS Cumulative Return
Cumulative Return
0.05
0.00
−0.05
−0.10
−0.15
−0.20
2025-11
2026-01
2026-03
2026-05
2026-07
2026-09
Date

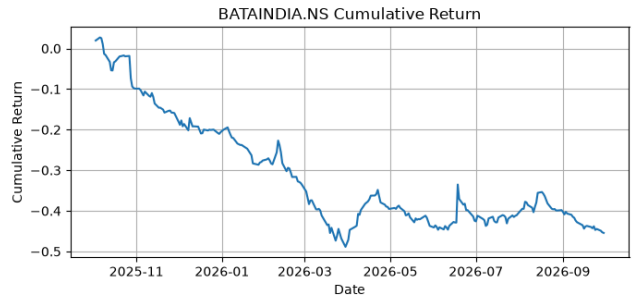

METROBRAND.NS Cumulative Return
Cumulative Return
0.00
−0.05
−0.10
−0.15
−0.20
−0.25
−0.30
2025-11
2026-01
2026-03
2026-05
2026-07
2026-09
Date

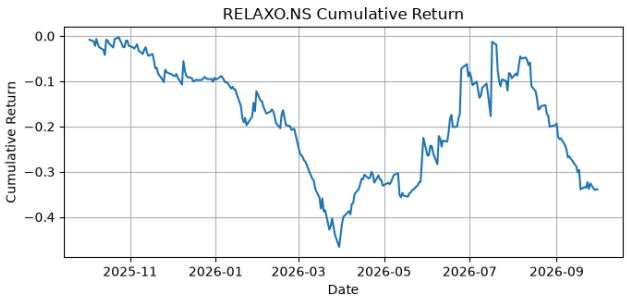


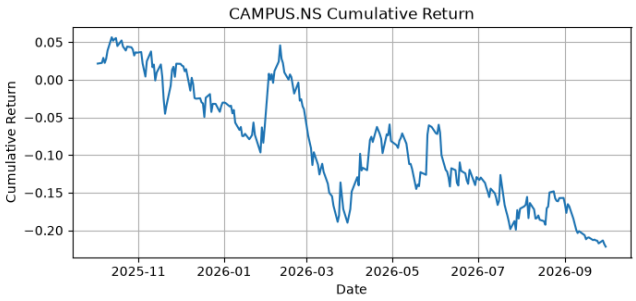
In [8]:
```
# Combined chart
plt.figure(figsize=(10, 6))
for symbol in cumulative_returns_df.columns:
    plt.plot(cumulative_returns_df.index, cumulative_returns_df[symbol],
plt.title("Cumulative Returns of Footwear Stocks")
plt.xlabel("Date")
plt.ylabel("Cumulative Return")
plt.legend()
plt.grid(True)
plt.show()
```

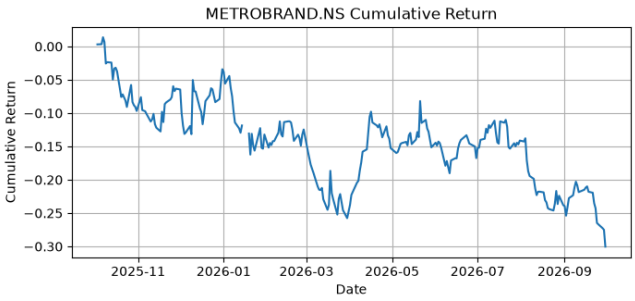


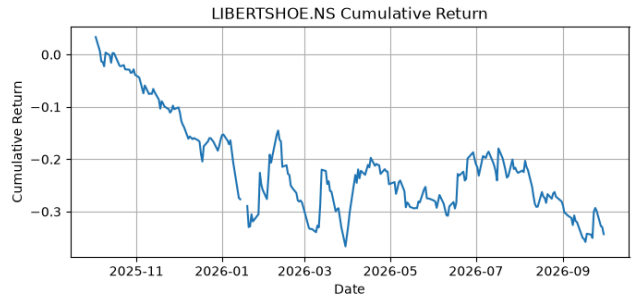
In [9]:
```
# Grid of subplots (3 rows x 2 columns)
plt.figure(figsize=(14, 10))
for i, symbol in enumerate(cumulative_returns_df.columns):
    plt.subplot(3, 2, i + 1)
    plt.plot(cumulative_returns_df.index, cumulative_returns_df[symbol])
    plt.title(f"{symbol} Cumulative Return")
    plt.xlabel("Date")
    plt.ylabel("Cumulative Return")
    plt.grid(True)
plt.tight_layout()
plt.show()
```











7. Key numbers for the written summary

In [10…

```
summary = pd.DataFrame({
    "Final cumulative return (%)": cumulative_returns_df.iloc[-1] * 100,
    "Average daily return (%)": returns_df.mean() * 100,
    "Volatility, std of daily return (%)": returns_df.std() * 100,
    "Best day (%)": returns_df.max() * 100,
    "Worst day (%)": returns_df.min() * 100,
}).round(2)
print(summary)
```

```
               Final cumulative return (%)  Average daily return (%)  \
BATAINDIA.NS                         -45.43                     -0.23
RELAXO.NS                            -33.91                     -0.13
CAMPUS.NS                            -22.17                     -0.09
METROBRAND.NS                        -30.03                     -0.13
LIBERTSHOE.NS                        -34.31                     -0.14

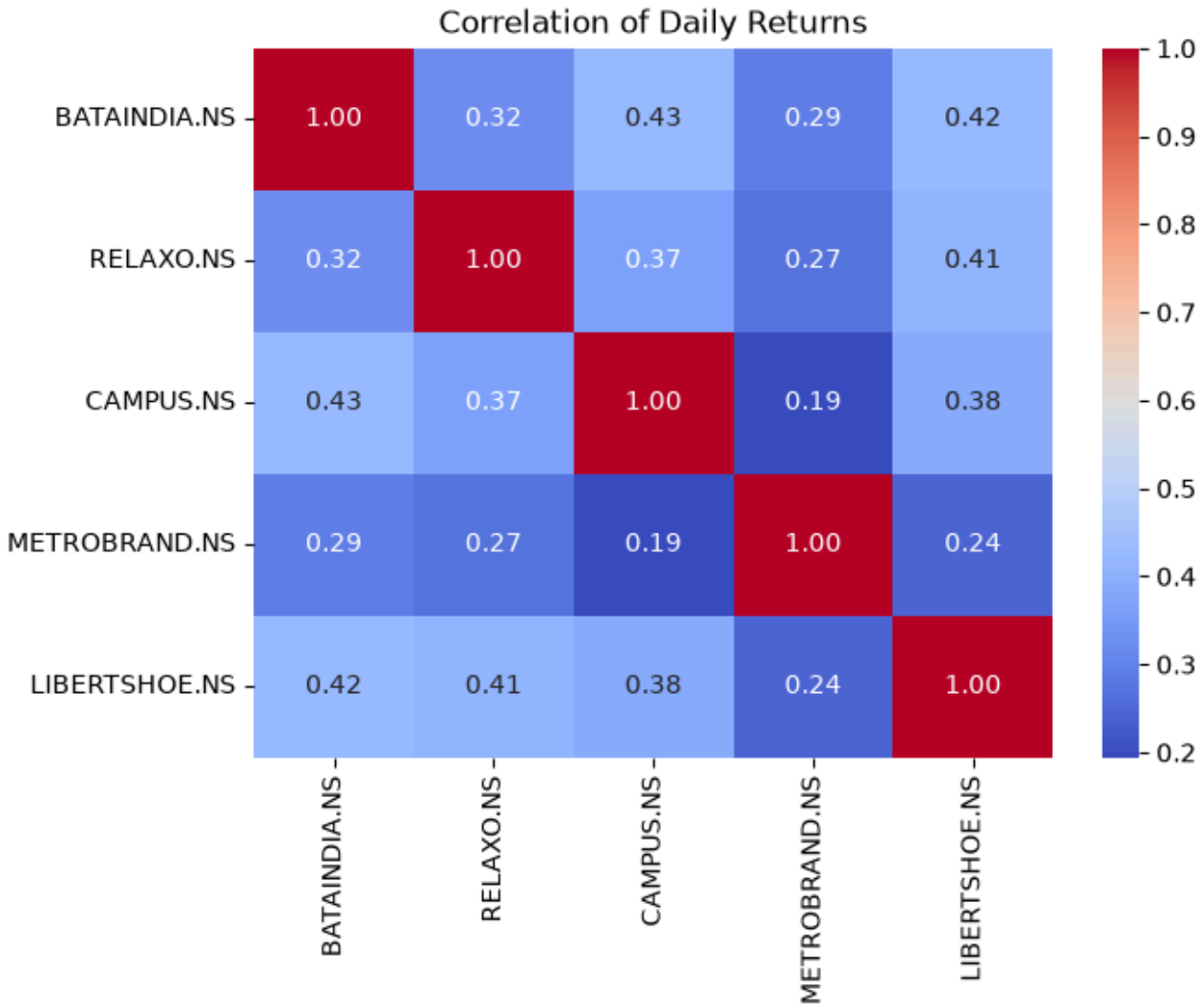
               Volatility, std of daily return (%)  Best day (%)  \
BATAINDIA.NS                                  1.94         16.42
RELAXO.NS                                     2.78         19.99
CAMPUS.NS                                     1.66          7.56
METROBRAND.NS                                 1.86          9.34
LIBERTSHOE.NS                                 2.52         11.37

               Worst day (%)
BATAINDIA.NS           -5.66
RELAXO.NS              -6.97
CAMPUS.NS              -4.56
METROBRAND.NS          -4.11
LIBERTSHOE.NS          -6.28
```

Extra analysis on the same data

In [11…

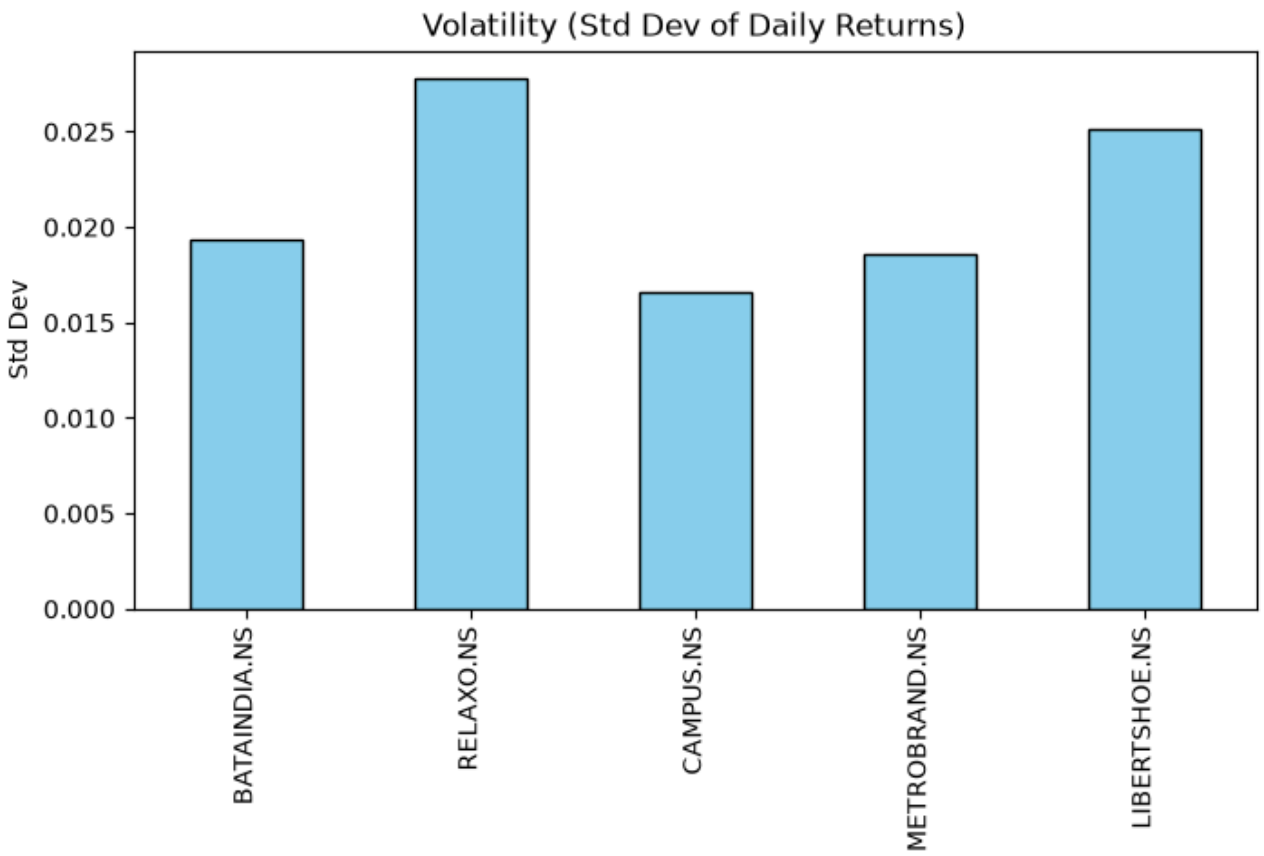
```python
# Correlation heatmap of daily returns
plt.figure(figsize=(7, 5))
sns.heatmap(returns_df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation of Daily Returns")
plt.show()
```

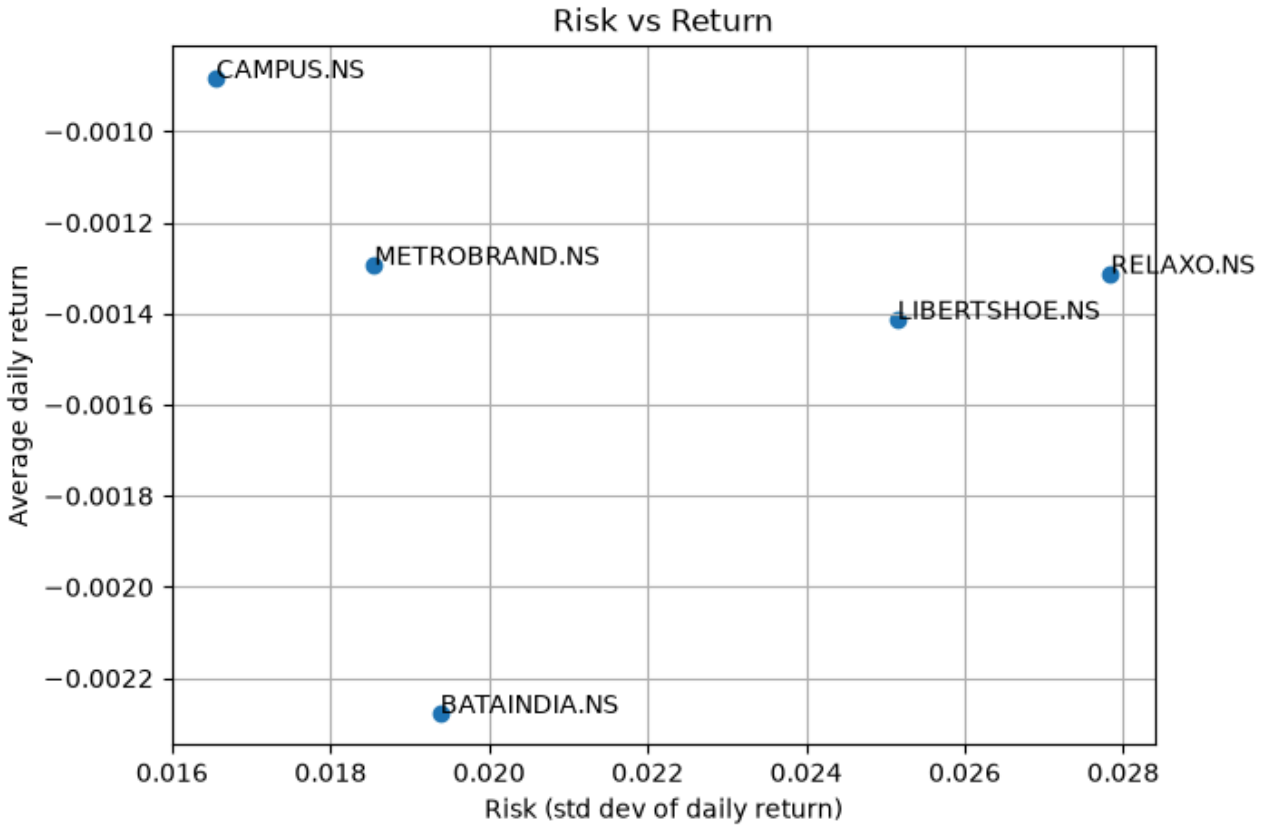


In [12…

```
# Volatility comparison (bar chart) and risk-return scatter
vol = returns_df.std()
avg = returns_df.mean()
vol.plot(kind="bar", color="skyblue", edgecolor="black", figsize=(8, 4))
plt.title("Volatility (Std Dev of Daily Returns)")
plt.ylabel("Std Dev")
plt.show()

plt.figure(figsize=(7, 5))
plt.scatter(vol, avg)
for s in vol.index:
    plt.annotate(s, (vol[s], avg[s]))
plt.xlabel("Risk (std dev of daily return)")
plt.ylabel("Average daily return")
plt.title("Risk vs Return")
plt.grid(True)
plt.show()
```


Volatility (Std Dev of Daily Returns)
Std Dev
0.025
0.020
0.015
0.010
0.005
0.000
BATAINDIA.NS
RELAXO.NS
CAMPUS.NS
METROBRAND.NS
LIBERTSHOE.NS

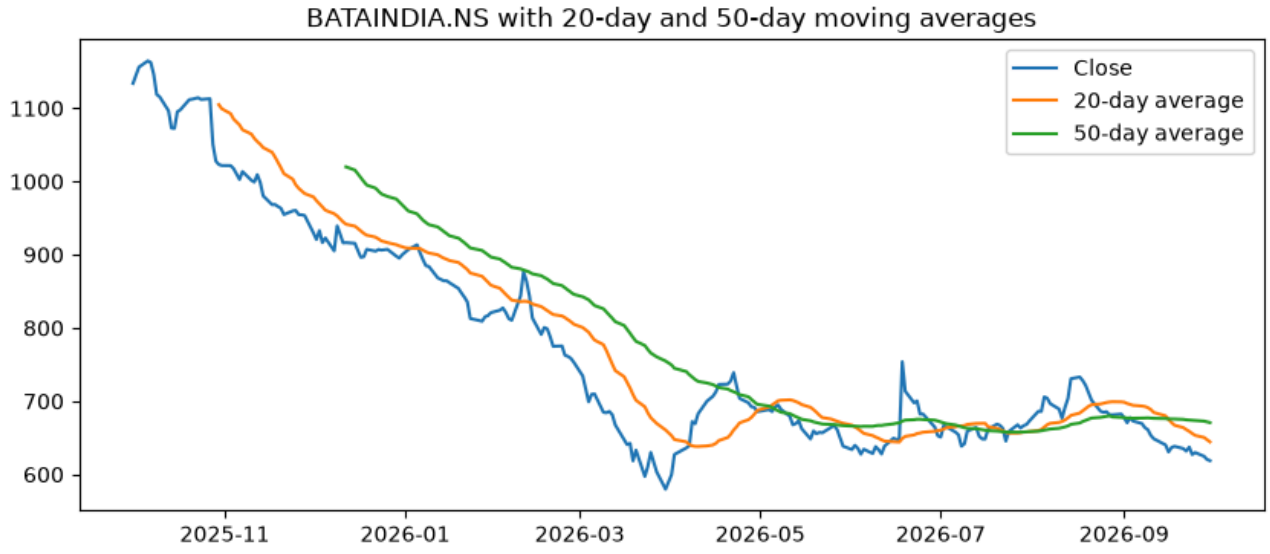

Risk vs Return
Average daily return
−0.0010
−0.0012
−0.0014
−0.0016
−0.0018
−0.0020
−0.0022
CAMPUS.NS
METROBRAND.NS
LIBERTSHOE.NS
RELAXO.NS
BATAINDIA.NS
0.016
0.018
0.020
0.022
0.024
0.026
0.028
Risk (std dev of daily return)

In [13…
```
# Sharpe ratio (annualised, assuming a risk-free rate of 7% a year; chang
rf_daily = 0.07 / 252
sharpe = (returns_df.mean() - rf_daily) / returns_df.std() * np.sqrt(252)
print(sharpe.round(2))

# Maximum drawdown: biggest fall from a previous peak
growth = 1 + cumulative_returns_df.fillna(0)
drawdown = growth / growth.cummax() - 1
print((drawdown.min() * 100).round(2))
```

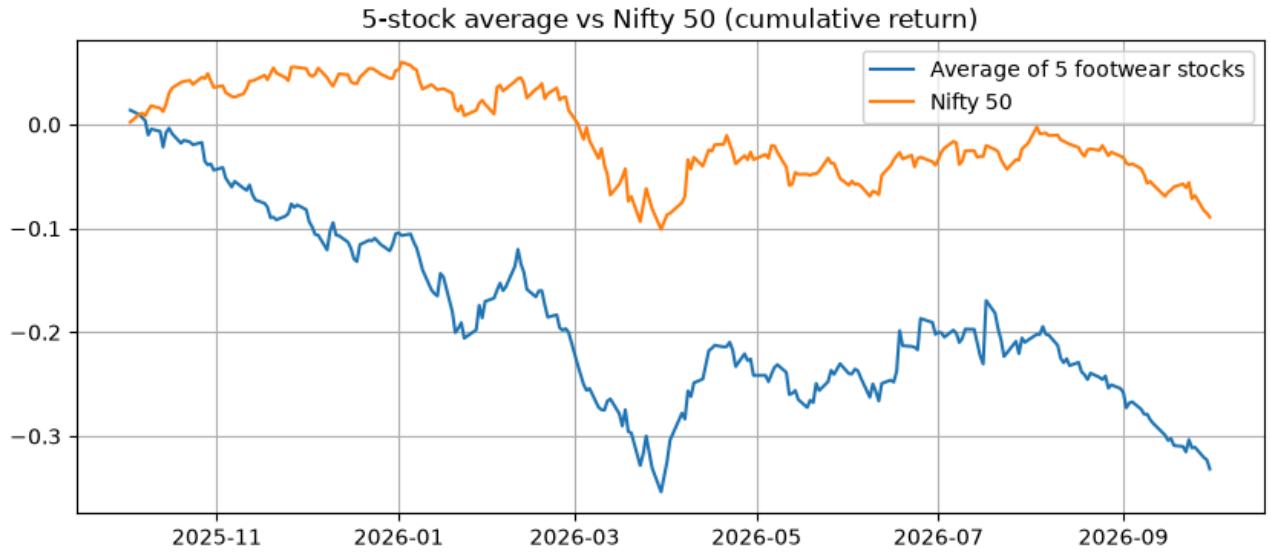
```
BATAINDIA.NS    -2.09
RELAXO.NS       -0.91
CAMPUS.NS       -1.11
METROBRAND.NS   -1.34
LIBERTSHOE.NS   -1.07
dtype: float64
BATAINDIA.NS    -50.22
RELAXO.NS       -46.61
CAMPUS.NS       -26.33
METROBRAND.NS   -30.98
LIBERTSHOE.NS   -38.71
dtype: float64
```

In [14…
```
# 20-day and 50-day moving averages on one stock's closing price (change
symbol = "BATAINDIA.NS"
price = stock_data[symbol]["Close"]
plt.figure(figsize=(10, 4))
plt.plot(price, label="Close")
plt.plot(price.rolling(20).mean(), label="20-day average")
plt.plot(price.rolling(50).mean(), label="50-day average")
plt.title(f"{symbol} with 20-day and 50-day moving averages")
plt.legend()
plt.show()
```



In [15…
```
# Benchmark: average cumulative return of the 5 stocks vs Nifty 50
nifty = yf.download("^NSEI", start=start_date, end=end_date, multi_level_
```

```
nifty_cum = (1 + nifty.pct_change()).cumprod() - 1
basket = cumulative_returns_df.mean(axis=1)
plt.figure(figsize=(10, 4))
plt.plot(basket, label="Average of 5 footwear stocks")
plt.plot(nifty_cum, label="Nifty 50")
plt.title("5-stock average vs Nifty 50 (cumulative return)")
plt.legend()
plt.grid(True)
plt.show()
```



Thankyou

In [ ]: